Course Overview

Sunglok Choi, Assistant Professor, Ph.D.
Dept. of Computer Science and Engineering, SEOULTECH
sunglok@seoultech.ac.kr | https://mint-lab.github.io/

Course Information

- When
  - Wednesday 14:00 – 16:50 in Korean
- Where
  - Offline: **Mirae Hall B104**
  - Online: **SEOULTECH e-Class (+ Zoom if necessary)**
- Lecturer: **Sunglok Choi**
  - E-mail: sunglok@seoultech.ac.kr
    - An online meeting is available if you request it in advance **via e-mail**.
  - Office: 327 Mirae Hall
- Textbook: **Course slides**
  - https://github.com/mint-lab/cv_tutorial
- Online References: MINT Lab's Know-Where
  - Please refer the *Computer Vision* category, especially with highlight

Course Objectives

- This course aims to you to enable to
  - Understand **the following <u>theories</u>** using Python, NumPy, and OpenCV
    - Image processing (image filtering) and color space
    - Camera projection models, camera calibration, and camera pose estimation
    - Planar homography and image transformation
    - CNN backbone networks and vision foundation model
    - Image classification and segmentation
    - Object recognition and tracking
  - Implement **your own <u>applications</u>** such as
    - Media arts
    - Augmented reality
    - Image rectification and stitching
    - Image classification
    - Pedestrian/vehicle detection

Understanding CV and Making My CV Project!

Motivation: Why Learn Computer Vision?

- Building computer vision background to understand papers

Preliminaries: Given an input video, we represent a scene as a collection of camera poses $\mathbf{T} \in \mathbb{SE}(3)^N$ and a set of square image patches $\mathbf{P}$ extracted from the video frames. Using $\mathbf{d}$ to represent inverse depth and $(\mathbf{x}, \mathbf{y})$ to represent pixel coordinates, we represent each patch as the $4 \times p^2$ homogeneous array

$$\mathbf{P}_k = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{1} \\ \mathbf{d} \end{pmatrix} \qquad \mathbf{x}, \mathbf{y}, \mathbf{d} \in \mathbb{R}^{1 \times p^2} \tag{1}$$

where p is the width of the patch. We assume a constant depth for the full patch, meaning that it forms a fronto-parallel plane in the frame from which it was extracted. Letting i denote the index of the *source frame* of the patch, i.e. the frame from which which patch $\mathbf{P}_k$ was extracted, we can reproject the patch onto another frame j

$$\mathbf{P}'_{kj} \sim \mathbf{K}\mathbf{T}_j\mathbf{T}_i{}^{-1}\mathbf{K}^{-1}\mathbf{P}_k. \tag{2}$$

taking $\mathbf{K}$ to be the 4×4 calibration matrix

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{3}$$

The pixel coordinates $\mathbf{x}' = (x', y')$ can be recovered by dividing by the third element. For the rest of the paper, we use the shorthand $\mathbf{P}'_{kj} = \omega_{ij}(\mathbf{T}, \mathbf{P}_k)$ to denote the reprojection of patch k onto frame j in terms of pixel coordinates.

Teed et al, "**D**eep **P**atch **V**isual **O**dometry", NeurIPS, 2023 Github

3. Method

Before delving into the details of our method, we introduce below the essential concept of pointmaps.

Pointmap. In the following, we denote a dense 2D field of 3D points as a *pointmap* $X \in \mathbb{R}^{W \times H \times 3}$. In association with its corresponding RGB image I of resolution $W \times H$, X forms a one-to-one mapping between image pixels and 3D scene points, *i.e.* $I_{i,j} \leftrightarrow X_{i,j}$, for all pixel coordinates $(i, j) \in \{1 \dots W\} \times \{1 \dots H\}$. We assume here that each camera ray hits a single 3D point, *i.e.* ignoring the case of translucent surfaces.

Cameras and scene. Given the camera intrinsics $K \in \mathbb{R}^{3 \times 3}$, the pointmap X of the observed scene can be straightforwardly obtained from the ground-truth depthmap $D \in \mathbb{R}^{W \times H}$ as $X_{i,j} = K^{-1}\left[iD_{i,j}, jD_{i,j}, D_{i,j}\right]^\top$. Here, X is expressed in the camera coordinate frame. In the following, we denote as $X^{n,m}$ the pointmap X^n from camera n expressed in camera m's coordinate frame:

$$X^{n,m} = P_m P_n^{-1} h\left(X^n\right) \tag{1}$$

with $P_m, P_n \in \mathbb{R}^{3 \times 4}$ the world-to-camera poses for images n and m, and $h : (x, y, z) \to (x, y, z, 1)$ the homogeneous mapping.

Wang et al, "**DUSt3R**: Geometric 3D Vision Made Easy", CVPR, 2024 Homepage

Motivation: Why Learn Computer Vision?

- Building computer vision background to understand papers

3.1 Formulation

We denote the input as $\mathcal{I} = \{\mathbf{I}_i\}_{i=1}^{N_v}$ with each $\mathbf{I}_i \in \mathbb{R}^{H\times W\times 3}$. For $N_v = 1$ this is a monocular image, and for $N_v > 1$ it represents a video or multi-view set. Each image has depth $\mathbf{D}_i \in \mathbb{R}^{H\times W}$, camera extrinsics $[\mathbf{R}_i \mid \mathbf{t}_i]$, and intrinsics $\mathbf{K}_i$. The camera can also be represented as $\mathbf{v}_i \in \mathbb{R}^9$ with translation $\mathbf{t}_i \in \mathbb{R}^3$, rotation quaternion $\mathbf{q}_i \in \mathbb{R}^4$, and FOV parameters $\mathbf{f}_i \in \mathbb{R}^2$. A pixel $\mathbf{p} = (u, v, 1)^\top$ projects to a 3D point $\mathbf{P} = (X, Y, Z, 1)^\top$ by

$$\mathbf{P} = \mathbf{R}_i\big(\mathbf{D}_i(u,v)\,\mathbf{K}_i^{-1}\mathbf{p}\big) + \mathbf{t}_i,$$

through which the underlying 3D visual space can be faithfully recovered.

Depth-ray representation. Predicting a valid rotation matrix $\mathbf{R}_i$ is challenging due to the orthogonality constraint. To avoid this, we represent camera pose implicitly with a per-pixel ray map, aligned with the input image and depth map. For each pixel $\mathbf{p}$, the camera ray $\mathbf{r} \in \mathbb{R}^6$ is defined by its origin $\mathbf{t} \in \mathbb{R}^3$ and direction $\mathbf{d} \in \mathbb{R}^3$: $\mathbf{r} = (\mathbf{t}, \mathbf{d})$. The direction is obtained by backprojecting $\mathbf{p}$ into the camera frame and rotating it to the world frame: $\mathbf{d} = \mathbf{R}\mathbf{K}^{-1}\mathbf{p}$. The dense ray map $\mathbf{M} \in \mathbb{R}^{H\times W\times 6}$ stores these parameters for all pixels. We do not normalize $\mathbf{d}$, so its magnitude preserves the projection scale. Thus, a 3D point in world coordinates is simply $\mathbf{P} = \mathbf{t} + \mathbf{D}(u,v)\cdot\mathbf{d}$. This formulation enables consistent point cloud generation by combining predicted depth and ray maps through element-wise operations.

Lin et al, "**Depth Anything 3**: Recovering the Visual Space from Any Views", arXiv, 2025 Homepage

C NDC ray space derivation

We reconstruct real scenes with "forward facing" captures in the normalized device coordinate (NDC) space that is commonly used as part of the triangle rasterization pipeline. This space is convenient because it preserves parallel lines while converting the z axis (camera axis) to be linear in disparity.

Here we derive the transformation which is applied to rays to map them from camera space to NDC space. The standard 3D perspective projection matrix for homogeneous coordinates is:

$$M = \begin{pmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \tag{7}$$

where n, f are the near and far clipping planes and r and t are the right and top bounds of the scene at the near clipping plane. (Note that this is in the convention where the camera is looking in the $-z$ direction.) To project a homogeneous point $(x, y, z, 1)^\top$, we left-multiply by M and then divide by the fourth coordinate:

$$\begin{pmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{n}{r}x \\ \frac{n}{t}y \\ \frac{-(f+n)}{f-n}z - \frac{-2fn}{f-n} \\ -z \end{pmatrix} \tag{8}$$

$$\text{project} \rightarrow \begin{pmatrix} \frac{n}{r}\frac{x}{-z} \\ \frac{n}{t}\frac{y}{-z} \\ \frac{(f+n)}{f-n} - \frac{2fn}{f-n}\frac{1}{-z} \end{pmatrix} \tag{9}$$

The projected point is now in normalized device coordinate (NDC) space, where the original viewing frustum has been mapped to the cube $[-1, 1]^3$.

Our goal is to take a ray $\mathbf{o} + t\mathbf{d}$ and calculate a ray origin $\mathbf{o}'$ and direction $\mathbf{d}'$ in NDC space such that for every t, there exists a new t' for which $\pi(\mathbf{o} + t\mathbf{d}) = \mathbf{o}' + t'\mathbf{d}'$ (where π is projection using the above matrix). In other words, the projection of the original ray and the NDC space ray trace out the same points (but not necessarily at the same rate).

Midenhall et al, "**NeRF**: Representing Scenes as Neural Radiance Fields for View Synthesis", ECCV, 2020 Homepage

(Tentative) Weekly Schedule

Week	Topics	Homework Assignments	
01	Course Overview / Introduction	-	
02	Image Editing: Learning OpenCV	Video Recorder	
03	Image Processing	Cartoon Rendering	
04	Color	-	
05	Image Formation	Camera Calibration	
06	Image Geometry	Camera Pose Estimation and AR	
07	~~Solving Problems~~ Image Correspondence	-	← Video Lecture (School Founding Day)
08	~~Image Correspondence~~ Solving Problems	Image Stitching	
09	Image Classification: CNN Backbone Networks	Transfer Learning	
10	Image Segmentation / Object Detection	*Term Project Proposal*	
11	*Midterm Examination*	-	
12	Toward Vision Foundation Models	-	
13	Object Tracking	-	
14	~~3D Vision~~ *Term Project Online Exhibition*	*Term Project Submission*	← Individual Meeting (Election Day)
15	~~*Term Project Online Exhibition*~~ 3D Vision	-	

Grading Policy

- **Examination (30%)**
  - Mid-term exam: 30% / No final exam!
- **Homework assignments (30%)**
  - Homework assignments will be given **almost every 1 or 2 weeks** (roughly 6 times).
  - Their deadline is generally **within roughly 1 or 2 weeks**.
- **Term project (30%)**
  - Topic: Making **your desired CV software or systems**.
  - You need to use **online Git repository** (e.g. Github, Bitbucket) as public or private.
  - You need to present your project using **`README.md`**.
  - You can organize a **team** or do your project **individually**. (I will look at your Git log.)
  - You can use other programming language and tools beyond this class.
- **Attendance (10%)**
  - Please let me know your absence due to public affairs **though e-mail** to be recognized as attendance.

Grading Policy

- **Grading**: As good as possible within the guideline of the school
  - Percentile
    - **Korean course: A+ 15%, A0 30%, B+ 50%, B0 70%**
    - English or 4th-year course: A+ 20%, A0 40%, B+ 60%, B0 80%
  - Average
    - **Korean course: Max 3.43**
    - English or 4th-year course: Max 3.60
  - Note) Your total score and rank will be open at the end of evalution.

    Please claim on your score within the given deadline.

Prerequisite

- **Python**
  - Note) Python Cheatsheet
- **NumPy**
  - Note) NumPy Cheatsheet
- **PyTorch**
  - Note) PyTorch Cheatsheet
- OpenCV (optional)
  - Note) OpenCV Cheatsheet

Prerequisite: Python

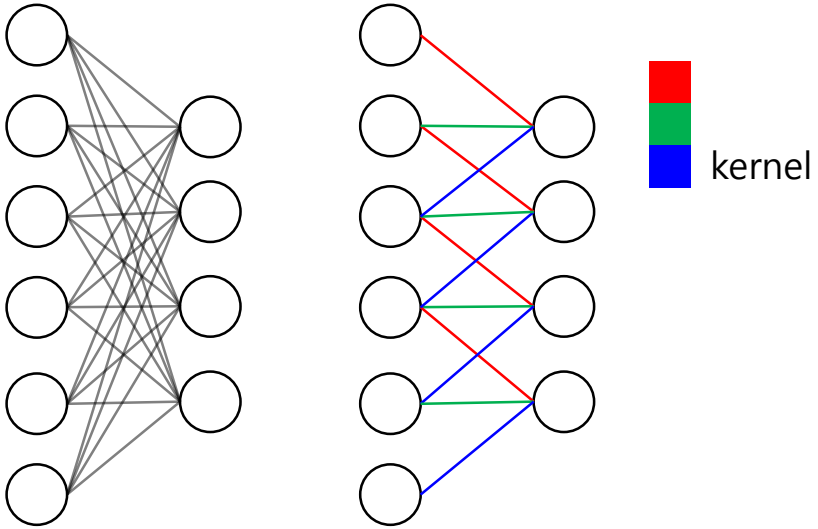
- Q) (Built-in constant) Are `None` and `False` same? (`None == False`) What is their difference?
- Q) (String) Do you know **f-string**? What is its advantage?
  - e.g. `f'My name is {name} and my next room is {room_no + 1:04d}'`
- Q) (Compound data) What is difference between **tuple** and **list**? What are **packing** and **unpacking**?
  - e.g. `(a, b) = (b, a)`
- Q) (Compound data) Can you understand **list comprehension**?
  - e.g. `sum([x**2 for x in range(4)])`
- Q) (Operators) What is the **membership operator, `in`**?
  - e.g. `1 not in range(0, 5, 2)`
- Q) (Flow control) What are falsy and trusy values?
  - e.g. `[False, 0, 0., None, '', (), [], {}, range(0), (None), (None,), [None], {None}]`
- Q) (Function) How many values does a Python function return? (e.g. 1 or 2 or …)

Prerequisite: NumPy

- Q) (Array) What is difference between **list** and **NumPy array**?
  - Both can concatenate Python data in multiple dimensions.
    - List: **Heterogeneous** data type / NumPy array: **Homogeneous** data type
      - Note) NumPy array is close to a concept of array in C/C++ and Java.
  - NumPy array provides
    - **Advanced indexing** (e.g. logical/fancy indexing)
    - **Advanced slicing** (e.g. column/subarray slicing)
    - **Arithmetic operations** (e.g. addition, multiplication)
    - **Linear algebra** (e.g. matrix multiplication, pseudo-inverse)
    - Useful utilities and tools (e.g. `load()`/`save()`, `loadtxt()`/`savetxt()`)

Prerequisite: PyTorch

- Q) (Tensor) What is difference between **NumPy array** and **PyTorch tensor**?
  - (From my point of view) PyTorch is an extention of NumPy with supports for
    - A n-dimensional array (a.k.a tensor) with **GPU acceleration** and **automatic differentiation**
    - **Useful modules for building and training neural networks**

- Q) (CNN) How many parameters of the following **FC layer** and **CNN layer**?

- Q) (CNN) What is advantage of CNN layers?